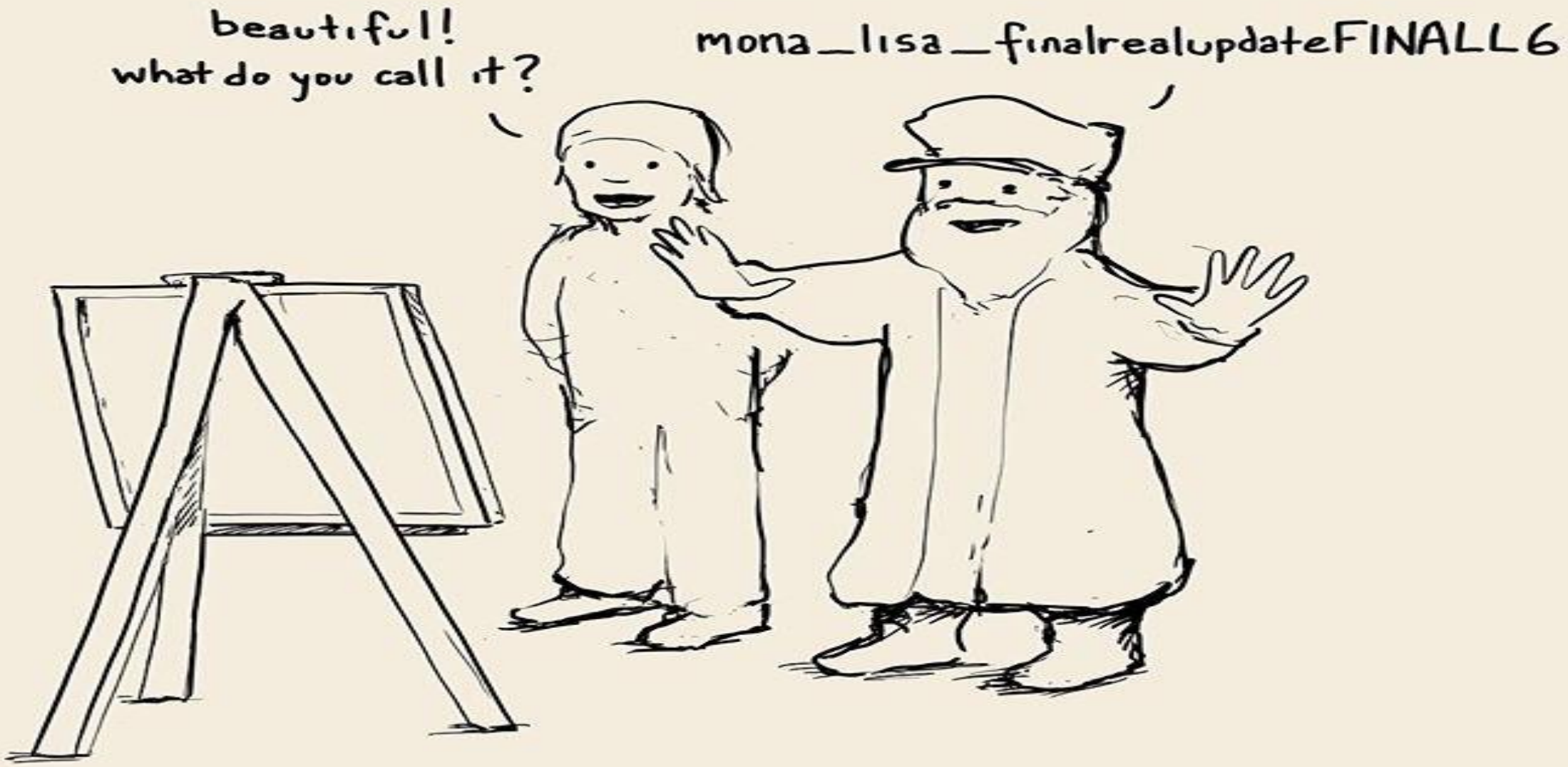
Version Control and Git

CSYE 6225 - Boston

Spring 2026

Tejas Parikh (t.parikh@northeastern.edu)

NORTHEASTERN.


beautiful!
what do you call it?
mona_lisa_finalrealupdateFINALL6

Learning Objectives

- Define version control and explain its fundamental purpose in software development
- Articulate the business and technical reasons why version control is essential for professional software development
- Compare Git with other version control systems and justify Git's dominance in the industry
- Set up Git on their local machine and perform basic repository operations
- Understand how version control integrates with cloud-native development workflows

Version Control

What is Version Control?

Version control is a system that records changes to files over time so that you can recall specific versions later.

It does the following:

- **Records changes**: Every modification is logged
- **Over time**: Creates a historical timeline of your project
- **Recall specific versions**: You can go back to any point in history

Core Concepts of Version Control

- **Repository (Repo)**: The database that stores all files and their complete history. Think of it as a project folder with superpowers.
- **Commit**: A snapshot of your project at a specific point in time. Each commit has a unique identifier, a message describing what changed, and metadata (who, when).
- **Branch**: A parallel version of your repository. Like making a copy where you can experiment without affecting the main project.
- **Merge**: Combining changes from different branches back together.

Types of Version Control Systems

- **Local Version Control**: Copies files into another directory (maybe with timestamps). Simple but error-prone. Example: manually creating backup folders like project_v1, project_v2, project_final, project_final_FINAL
- **Centralized Version Control Systems (CVCS)**: Single server contains all versioned files. Clients check out files from central place. Examples: CVS, Subversion (SVN), Perforce
  - Advantage: Everyone knows what everyone else is doing
  - Disadvantage: Single point of failure - if server goes down, no one can collaborate or save versioned changes
- **Distributed Version Control Systems (DVCS)**: Every clone is a full backup of the repository. Examples: Git, Mercurial
  - Every developer has a complete copy of the repository history
  - No single point of failure
  - Can work offline and sync later
  - This is why Git dominates in cloud environments - resilience and flexibility

Why Should Software Developers Use Version Control?

Collaboration Without Chaos

- Modern software is rarely built alone. Even in small startups, you'll work with others.
- Version control provides:
  - **Parallel development**: Multiple developers can work on different features simultaneously
  - **Conflict resolution**: When two people modify the same file, version control helps identify and resolve conflicts
  - **Code review**: Changes can be reviewed before being incorporated
  - **Attribution**: You always know who changed what and when

Safety Net for Your Code

- **Rollback capability**: Made a mistake? Revert to a previous working version instantly
- **Recovery**: Accidentally deleted a file? It's still in the version history
- **Experimentation**: Create a branch, try something crazy. If it works, merge it. If not, delete the branch. No risk to the main codebase.

Complete History and Audit Trail

Every commit records:

- **Who**: Author information
- **What**: The exact changes (diffs)
- **When**: Timestamp
- **Why**: Commit message explaining the reason for the change

Enables DevOps and CI/CD

- Version control is the **foundation** of modern DevOps practices
- CI/CD pipelines are triggered by version control events:
  - Push to a branch → Run automated tests
  - Merge to main → Build and deploy to staging
  - Tag a release → Deploy to production
- **Infrastructure as Code**: With tools like Terraform (which we'll cover later), even your infrastructure configurations are version controlled

Why Git?

A Brief History of Git

- Created by Linus Torvalds in 2005 (yes, the Linux creator)
- Built to manage Linux kernel development - one of the largest open-source projects
- Previous tool (BitKeeper) had licensing issues, so Linus created his own
- Design goals:
  - **Speed**: Operations should be fast
  - **Simple design**: Easy to understand conceptually
  - **Strong support for non-linear development**: Thousands of parallel branches
  - **Fully distributed**: Everyone has full history
  - **Able to handle large projects**: Linux kernel has millions of lines of code

Fun Fact: "Git was named by Linus Torvalds. In British slang, 'git' means an unpleasant person. Linus said he names his projects after himself - first Linux, then Git."

Git's Architecture - What Makes It Different

- **Snapshot-based, not delta-based**:
  - Most VCS store changes as deltas (differences from previous version)
  - Git stores snapshots of the entire project state
  - If a file hasn't changed, Git stores a reference to the previous identical file
  - This makes branching and merging incredibly fast
- **Nearly every operation is local**:
  - Browse history? Local
  - Commit changes? Local
  - View diffs? Local
  - Only push/pull/fetch require network access
  - This means you can work on a plane with no internet
- **Integrity**:
  - Everything is check summed (SHA-1 hash)
  - Impossible to change contents without Git knowing
  - You'll see commits referenced by hashes like a1b2c3d4

The Git Ecosystem - Why Git Really Won

- **Hosting platforms**:
  - **GitHub**: Largest code hosting platform, acquired by Microsoft for $7.5 billion
  - **GitLab**: Full DevOps platform, self-hosted option popular in enterprises
  - **Bitbucket**: Atlassian's offering, integrates with Jira
- **Integrations everywhere**:
  - Every IDE has Git integration
  - CI/CD tools (Jenkins, GitHub Actions, GitLab CI)
  - Project management tools
  - Code review tools
  - Deployment platforms
- **Open Source community**:
  - Virtually all open source uses Git
  - Contributing to open source requires Git knowledge
  - Learning resources are abundant

Getting Started with Git

Installing Git

- **Linux (Ubuntu/Debian)**:

```
sudo apt update
sudo apt install git
```

- **Linux (RHEL/CentOS/Fedora)**:

```
sudo dnf install git
# or for older systems
sudo yum install git
```

- **macOS**:
  - Install Xcode Command Line Tools: `xcode-select --install`
  - Or use Homebrew: `brew install git`
- **Windows**:
  - Download from git-scm.com
  - Git Bash provides a Unix-like terminal
  - Or use Windows Subsystem for Linux (WSL) - recommended for this course
- **Verify installation**:

```
git --version
```

Initial Git Configuration

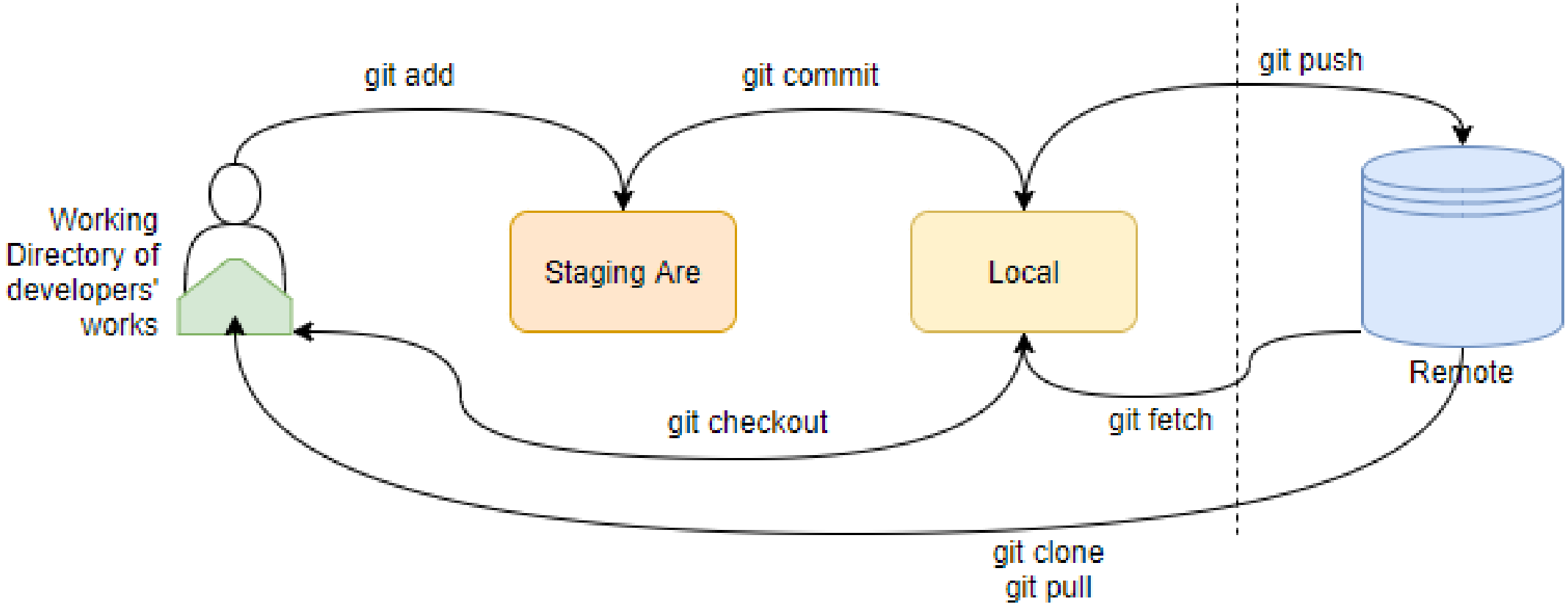
```
# Set your identity (REQUIRED)
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Set default branch name (recommended)
git config --global init.defaultBranch main

# Set default editor (optional but helpful)
git config --global core.editor "vim"
# or "code --wait" for VS Code

# View your configuration
git config --list
```

The Three States of Git

```
Working Directory → Staging Area → Repository
     (edit)       →   (stage)   →  (commit)
```

- **Working Directory**: Your actual files that you edit
- **Staging Area (Index)**: A holding area for your next commit. Lets you craft commits carefully.
- **Repository (.git)**: The database of all commits and history
- **Why a staging area?**
  - Commit related changes together
  - Leave unrelated changes for a different commit
  - Review what you're about to commit

Git Flow


git add
git commit
git push
Working Directory of developers' works
Staging Are
Local
Remote
git checkout
git fetch
git clone
git pull

Essential Git Commands

I DONT KNOW ABOUT
THEM FANCY GIT COMMANDS
makeameme.org

Creating/Cloning

- git init - Create new repository
- git clone <url> - Copy existing repository

Basic Workflow

- git status - Check current state (use frequently!)
- git add <file> - Stage changes
- git add . - Stage all changes
- git commit -m "message" - Create commit
- git log - View commit history
- git log --oneline - Compact history view

Best Practice - Commit Messages

- Write clear, descriptive messages
- Use imperative mood: "Add feature" not "Added feature"
- First line: brief summary (50 chars or less)
- If needed, blank line then detailed explanation

Example good commit messages:

- "Fix authentication bug in login handler"
- "Add user profile page with avatar support"
- "Refactor database connection pooling"

Branching

- git branch - List branches
- git branch <name> - Create branch
- git checkout <branch> - Switch branches
- git checkout -b <name> - Create and switch (shortcut)
- git merge <branch> - Merge branch into current branch

Remote Operations

- git remote add origin <url> - Add remote repository
- git push -u origin main - Push to remote (first time)
- git push - Push subsequent commits
- git pull - Fetch and merge from remote
- git fetch - Fetch without merging

Connecting to GitHub

```
# Check for existing keys
ls -la ~/.ssh

# Generate new key if needed
ssh-keygen -t ed25519 -C "your.email@example.com"

# Start SSH agent and add key
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Copy public key
cat ~/.ssh/id_ed25519.pub
# Add this to GitHub → Settings → SSH Keys

# Test connection
ssh -T git@github.com
```

Summary

Pull before push

“git pull a day keeps the conflicts away”

Write meaningful messages

Your future self will thank you

COMMENT	DATE
CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
ENABLED CONFIG FILE PARSING	9 HOURS AGO
MISC BUGFIXES	5 HOURS AGO
CODE ADDITIONS/EDITS	4 HOURS AGO
MORE CODE	4 HOURS AGO
HERE HAVE CODE	4 HOURS AGO
AAAAAAAA	3 HOURS AGO
ADKFJSLKDFJSDKLFJ	3 HOURS AGO
MY HANDS ARE TYPING WORDS	2 HOURS AGO
HAAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

Commit often

Small, focused commits are easier to review and debug

GIT MERGE

Git Best Practices

- **Use branches**: Keep main/master stable, develop in feature branches
- **Never commit secrets**: No passwords, API keys, or credentials in version control
- **Review before committing**: Use git diff and git status
- **Keep repositories clean**: Use .gitignore for build artifacts, dependencies

Forking Workflow

https://www.atlassian.com/git/tutorials/comparing-workflows/forking-workflow

Additional Resources

See course lecture page on Canvas for supplementary materials and documentation links.

LVX
VERITAS
VIRTVS